

NeuroClip: A Context-Aware Multimodal Video Processing System for Semantic Clip Retrieval

Ali Javed, Asad Sardar

Department of Computer Science, National University of Modern Languages, Faisalabad

Supervised by: Subhan Arif

Abstract

The explosive growth of long-form digital video content in educational, corporate, and media domains has created a critical demand for intelligent retrieval systems that surpass conventional keyword-based search. This paper presents **NeuroClip**, a full-stack, context-aware video processing platform that transforms static video archives into dynamically queryable knowledge bases. The system fuses two information modalities – Automated Speech Recognition (ASR) transcripts generated via AssemblyAI and Optical Character Recognition (OCR) text extracted from sampled frames via EasyOCR – into enriched sentence-level units, which are then encoded into dense semantic vector embeddings using the **all-MiniLM-L6-v2** sentence-transformer model. A windowed retrieval pipeline, augmented by optional cross-encoder re-ranking (**cross-encoder/ms-marco-MiniLM-L-6-v2**), identifies, scores, and merges the most contextually relevant transcript segments. FFmpeg then extracts precise playable clips from those segments. Additional modules handle query-driven sensitive content blurring (YOLOv8-based face, ID, and license plate redaction) and codec-aware video compression (H.265/HEVC). A persistent caching architecture keyed by unique job identifiers eliminates redundant re-processing, enabling sub-second cached query latency and full end-to-end pipeline execution within minutes on standard consumer hardware. Experimental evaluation across 50 structured query cases spanning mathematics, computer science, AI, and cybersecurity demonstrates strong semantic retrieval precision. The system is deployed as a Progressive Web Application with a React/TypeScript frontend and a FastAPI/Python backend, with vector storage and authentication managed by Supabase (PostgreSQL + pgvector).

Keywords: Semantic video retrieval, sentence embeddings, multimodal fusion, ASR, OCR, vector search, clip extraction, privacy-preserving video processing, FFmpeg, FastAPI

1. Introduction

Video has emerged as the dominant medium for knowledge dissemination across educational platforms, corporate intranets, and social networks. Cisco estimates that video constitutes over 82% of all internet traffic, and platforms such as YouTube, Coursera, and enterprise video repositories collectively host billions of hours of recorded content [1]. Despite this abundance, the ability to locate specific information within long-form video remains fundamentally inadequate. A student searching for a precise explanation of backpropagation within a two-hour machine learning lecture, or a corporate analyst attempting to locate a specific policy discussion within recorded board meetings, must typically resort to manual scrubbing – a process that is laborious, error-prone, and cognitively demanding.

Conventional video search systems address this problem through metadata tagging (titles, descriptions, closed captions) or basic keyword matching against automatic transcripts. These approaches exhibit three well-documented failure modes. First, **semantic gap**: keywords fail to capture intent – a query for "how

momentum is conserved" will not retrieve a segment that explains "the total impulse before and after collision remains identical." Second, **fragmentation**: even when transcript-based search returns a hit, the system typically delivers a short, decontextualized snippet that lacks the surrounding context needed to understand the answer. Third, **computational redundancy**: existing pipelines re-process the same video for every new search query, wasting significant GPU and CPU cycles on repeat transcription and embedding computation.

NeuroClip addresses all three failure modes through a unified, intent-aware pipeline. Its key contributions are:

1. **Multimodal fusion** of ASR speech transcripts and frame-level OCR visual text, producing enriched sentence units that capture both spoken and displayed information.
2. **Dense vector semantic search** using pre-computed sentence embeddings, enabling natural-language queries to retrieve semantically relevant segments rather than keyword matches.
3. **Windowed retrieval with optional cross-encoder re-ranking**, assembling coherent, contextually complete clips rather than fragmented snippets.
4. **Job-keyed persistent caching** of embeddings and metadata, eliminating redundant processing for subsequent queries on the same video.
5. **Query-driven privacy redaction** via YOLOv8 object detection to blur faces, ID cards, and license plates prior to clip delivery.
6. **Codec-aware video compression** pipeline for lightweight clip distribution.

The remainder of this paper is organized as follows. Section 2 reviews related literature. Section 3 describes the system architecture and methodology. Section 4 details the experimental setup and evaluation. Section 5 presents results and discussion. Section 6 outlines limitations and future work. Section 7 concludes the paper.

2. Related Work

2.1 Semantic Sentence Embeddings

Reimers and Gurevych (2019) introduced **Sentence-BERT (SBERT)**, adapting pre-trained BERT into a Siamese and triplet network architecture to produce semantically meaningful, fixed-size sentence embeddings [2]. Prior to SBERT, computing semantic similarity for 10,000 sentences using raw BERT required approximately 65 hours due to quadratic cross-attention costs; SBERT reduced this to approximately 5 seconds while maintaining strong performance on STS benchmarks. NeuroClip builds directly on this foundation, employing the **all-MiniLM-L6-v2** distillation of SBERT as its primary encoder. A notable limitation acknowledged in the SBERT work is that Bi-Encoder representations, while fast, can miss fine-grained query-document interactions – a limitation NeuroClip partially mitigates through its optional cross-encoder re-ranking stage.

2.2 Cross-Encoder Re-ranking for Passage Retrieval

Nogueira and Cho (2019) demonstrated that feeding query and candidate passage jointly into a BERT Cross-Encoder significantly improves retrieval precision over BM25 and Bi-Encoder baselines on datasets such as MS MARCO [3]. The cross-attention mechanism allows complete interaction modeling between query terms and document tokens. The principal drawback is high computational latency: since documents cannot be pre-encoded, each query-document pair requires a full forward pass. NeuroClip employs a two-stage architecture – fast Bi-Encoder retrieval for candidate selection, followed by Cross-Encoder re-ranking on a small shortlist – to balance precision and latency.

2.3 Transformer Architecture Foundations

Vaswani et al. (2017) introduced the Transformer architecture based entirely on self-attention mechanisms, eliminating recurrence and convolution [4]. This work underpins both the ASR models (Whisper-style architectures) and the sentence embedding models used in NeuroClip. A well-known limitation is the quadratic complexity with respect to sequence length, which motivates the windowed segmentation approach adopted in this work for processing long video transcripts.

2.4 Video Summarization with Deep Learning

Apostolidis et al. (2021) provide a comprehensive survey of deep learning approaches to video summarization, identifying key-frame selection, temporal segmentation, and attention-based highlight detection as dominant paradigms [5]. Vora et al. (2025) specifically examine AI-driven video summarization for content retrieval, highlighting the efficiency advantages of embedding-based approaches over earlier frequency-based methods [1]. NeuroClip adopts a query-conditioned summarization strategy rather than unsupervised highlight extraction, making it particularly suited to information retrieval use cases where user intent is explicit.

2.5 Privacy-Preserving Video Processing

Plaud and Lisani (2024) evaluate two deep learning solutions for automatic face blurring in videos, demonstrating that detection-based blurring substantially outperforms naive frame-level blurring in preserving non-sensitive regions [6]. Nemavhola et al. (2025) survey deep learning techniques for face recognition that inform modern detection architectures including YOLOv8 [7]. NeuroClip extends this to a multi-class blurring module detecting faces, license plates, and ID documents.

2.6 Video Compression

Sullivan and Wiegand (2005) established the foundational principles of hybrid video compression, leading to H.264/AVC and subsequently H.265/HEVC standards [8]. NeuroClip leverages FFmpeg's `libx265` (CPU) and `hevc_nvenc` (NVIDIA GPU) encoders for codec-aware compression with automatic hardware fallback.

2.7 Existing Systems and Their Limitations

System	Year	Strength	Limitation
CapCut	2024	Fast manual editing; face blur available	No automatic face detection
EasyMosaic	2022	Strong blur; preview before apply	Slow tracking; no auto-detection
Summarify	2021	Quick YouTube summarization	Limited to YouTube; no custom upload
Notta	2020	Accurate ASR; summarization notes	Limited language support; noise-sensitive
CNN Sensitive Detection [9]	2021	High privacy accuracy	Degraded performance under occlusion
RT Neural Compression [10]	2020	Qualityâ€”size balance	Requires GPU; low-resource underperformance

Collectively, the existing landscape reveals a fragmented ecosystem: transcription tools lack retrieval, retrieval tools lack semantic understanding, blurring tools lack automation, and no single system combines all of these capabilities into an end-to-end pipeline. NeuroClip fills this gap.

3. System Architecture and Methodology

3.1 Overview

NeuroClip is a full-stack Progressive Web Application (PWA) with a modular processing pipeline. Figure 1 illustrates the high-level architecture:

```

User Upload/URL
  â†“
React Frontend (Vite + TypeScript + TailwindCSS)
  â†“
FastAPI Backend
  â”€â”€â”€ AssemblyAI (ASR Transcription)
  â”€â”€â”€ EasyOCR (Frame-level OCR)
  â”€â”€â”€ Sentence Fusion & Embedding (all-MiniLM-L6-v2)
  â”€â”€â”€ Supabase (PostgreSQL + pgvector)
  â†“
Query Input â†’ Retriever + Cross-Encoder Re-ranker
  â†“
FFmpeg Clip Extraction â†’ Blurring Module â†’ Compression
  â†“
Ranked Playable Clips + Summaries â†’ Frontend
  
```

Figure 1: NeuroClip end-to-end pipeline architecture.

3.2 Video Ingestion

The system accepts video input through three API endpoints:

- `POST /upload-video` â€” local file upload (MP4, MOV, MKV, WebM)
- `POST /upload-via-url` â€” URL-based download via `yt-dlp`
- `POST /upload-and-search` â€” combined ingestion and immediate query

Upon receipt, a unique **job identifier** (`job_id`) is generated using UUID4. This identifier serves as the primary key linking the video file, its transcript, sentence embeddings, and processing history throughout the system lifecycle. Video files are stored in Supabase Storage; metadata is written to the `videos` table in PostgreSQL.

3.3 Automated Speech Recognition (ASR) via AssemblyAI

Audio is extracted from the video and submitted to AssemblyAI's transcription API, which returns an SRT-format transcript with word-level timestamps. The SRT output is parsed into sentence-level units, each carrying:

- `text`: the sentence string
- `start_time`: start timestamp in seconds

- `end_time`: end timestamp in seconds

AssemblyAI was selected over local Whisper models due to its superior accuracy on conversational and lecture-style audio, its robust handling of speaker diarization, and its negligible latency overhead compared to the subsequent embedding computation step.

3.4 OCR-Based Visual Signal Enrichment

To capture information displayed on-screen – such as slide text, whiteboard equations, code snippets, and captions – frames are sampled at a fixed interval (default: every 3 seconds). EasyOCR processes each sampled frame and extracts visible text regions. OCR snippets are then merged into the nearest transcript sentence by timestamp proximity. If no transcript sentence exists within a configurable threshold, the OCR text is inserted as a synthetic visual sentence. This fusion step ensures that search queries referencing on-screen content (e.g., "show me the slide about hash collision") can be semantically matched.

3.5 Sentence-Level Embedding Generation

Each enriched sentence unit (speech text + merged OCR context) is encoded into a 384-dimensional dense vector using the `sentence-transformers/all-MiniLM-L6-v2` model. This model was selected for its optimal trade-off between embedding quality (strong STS benchmark scores), inference speed (suitable for CPU inference), and memory footprint (~80MB). An optional OpenCLIP fallback is available for visual-semantic embedding when image-level retrieval is required.

The embedding computation step processes sentences in batches of 32 to maximize throughput. On a standard workstation (Intel Core i7, 16GB RAM, no GPU), a 10-minute lecture video produces approximately 120–200 sentence units and completes embedding generation in under 90 seconds.

Processed artifacts are persisted in two forms:

- **Local JSON files:** `.v4.json` (enriched sentence metadata) and `.embeddings.json` (sentence vectors), stored keyed by `job_id`
- **Supabase records:** `video_embeddings` and `video_sentence_embeddings` tables with `pgvector`-typed columns, enabling server-side vector similarity search

3.6 Retrieval Pipeline

3.6.1 Query Embedding

When a user submits a natural language query, the query string is encoded in the same 384-dimensional embedding space using the identical `all-MiniLM-L6-v2` encoder. This ensures semantic alignment between query and document representations.

3.6.2 Candidate Retrieval – Two-Stage Strategy

NeuroClip implements a two-stage retrieval strategy:

Primary route (LLM-assisted): The system leverages a Large Language Model (LLM) to perform intelligent segment reasoning, allowing it to understand multi-hop or composite queries (e.g., "explain the concept introduced after the motivation slide"). The LLM receives the top-k candidate sentences and produces a reasoning-grounded selection.

Fallback route (cosine similarity): When LLM assistance is unavailable (API timeout, quota exceeded), a cosine similarity search over the pgvector store retrieves the top-k most similar sentence embeddings. Supabase's RPC-based `match_video_embeddings` function executes this efficiently at the database layer.

3.6.3 Windowed Retrieval

Rather than retrieving individual sentences, NeuroClip employs a **windowed retrieval** approach. A window of `W` consecutive sentences is treated as a single retrieval unit. Each window is represented by the mean pooling of its constituent sentence embeddings. The window stride `S` controls the overlap between adjacent windows. This design ensures that retrieved segments contain sufficient contextual information before and after the peak relevance point.

Configurable parameters exposed to the user:

- `window_size` (default: 5 sentences)
- `stride` (default: 2 sentences)
- `min_clip_duration` (default: 10 seconds)
- `max_clip_duration` (default: 120 seconds)

3.6.4 Cross-Encoder Re-ranking

The top-N candidate windows are optionally submitted to a cross-encoder re-ranking step using `cross-encoder/ms-marco-MiniLM-L-6-v2`. The query and the concatenated window text are jointly encoded; the relevance score from the `[CLS]` token classification head replaces the initial cosine similarity score. If the re-ranker model is unavailable (model load failure, memory constraint), the system gracefully falls back to embedding-only ranking with a toast notification to the user.

3.6.5 Neighbor Merging

Adjacent highly-scored windows are merged into contiguous temporal segments using a neighbor merging algorithm. Two windows are merged if their temporal gap falls below a configurable threshold (default: 2 seconds). This prevents the delivery of multiple overlapping clips representing the same discussion thread and produces coherent, playable segments with natural entry and exit points.

3.7 FFmpeg Clip Extraction

Merged temporal segments are passed to the clip extraction module. FFmpeg is invoked programmatically to cut the original video file at the specified `[start_time, end_time]` boundaries with configurable padding margins (default: ± 1.5 seconds). The extraction uses stream-copy mode where possible (no re-encoding) to minimize latency. For blurred or compressed outputs, re-encoding is performed with the H.264 codec as the baseline format. Extracted clips are stored in Supabase Storage and URLs are returned in the API response payload.

3.8 Sensitive Content Blurring Module

NeuroClip includes a query-driven, privacy-aware blurring module. The user provides a natural language blurring instruction (e.g., "blur all faces in this segment" or "hide license plates between 2:30 and 4:00") and an optional time range.

The blurring workflow:

1. **Instruction parsing:** The NLP layer identifies the object type to blur (face, ID card, license plate) and the temporal range from the natural language instruction.
2. **Object detection:** YOLOv8 processes sampled frames within the specified range, detecting bounding boxes for the specified object class with confidence thresholding.
3. **Blur mask application:** A Gaussian blur filter is applied to each detected bounding box region-of-interest (ROI) on a per-frame basis. The blur kernel size is proportional to the bounding box area.
4. **Temporal propagation:** Detected regions are tracked across consecutive frames using bounding box interpolation, preventing flicker when the model misses a detection on an intermediate frame.
5. **Output rendering:** FFmpeg re-encodes the processed frames into the output clip, preserving non-sensitive regions at original quality.

3.9 Video Compression Module

The compression module (`POST /compress-video`) provides a practical post-processing step for distribution-ready clip delivery:

- **Codec selection:** NVIDIA GPU (`hevc_nvenc`) is preferred when available; CPU `libx265` serves as the fallback.
- **Resolution capping:** Output resolution is capped at 1280×720 (720p) by default to control bitrate.
- **Quality/size trade-off:** CRF (Constant Rate Factor) parameter controls the quality-size balance. Lower CRF values preserve quality; higher values maximize compression.
- **Output statistics:** The API response includes original size, compressed size, reduction percentage, and processing time.

Across the defined compression profiles (30s/360p to 1200s/1080p), the system achieves 60–78% file size reduction while maintaining perceptually acceptable quality for educational content delivery.

3.10 System Modules Summary

Module	Technology	Purpose
User Auth	Supabase Auth (email + password)	Secure access; row-level data isolation
Video Upload	FastAPI + Supabase Storage + yt-dlp	Local file and URL-based ingestion
ASR Transcription	AssemblyAI	Time-aligned speech-to-text
OCR Enrichment	EasyOCR + OpenCV	Frame-level visual text extraction
Embedding Generation	sentence-transformers (MiniLM-L6-v2)	384-dim dense semantic vectors
Vector Storage	Supabase + pgvector	Persistent embedding store
Semantic Retrieval	Cosine similarity + LLM routing	Top-k candidate selection
Re-ranking	cross-encoder/ms-marco-MiniLM-L-6-v2	Precision refinement

Module	Technology	Purpose
Clip Extraction	FFmpeg	Precise temporal clip cutting
Blurring	YOLOv8 + Gaussian filter	Privacy-preserving redaction
Compression	FFmpeg H.265/HEVC	Lightweight distribution output

4. System Design and Modeling

4.1 Use Case Model

The primary actor in NeuroClip is the **End User**, who interacts with the system through a browser-based interface. The system boundary encompasses the React frontend, FastAPI backend, AI processing pipeline, and Supabase data layer.

Primary use cases:

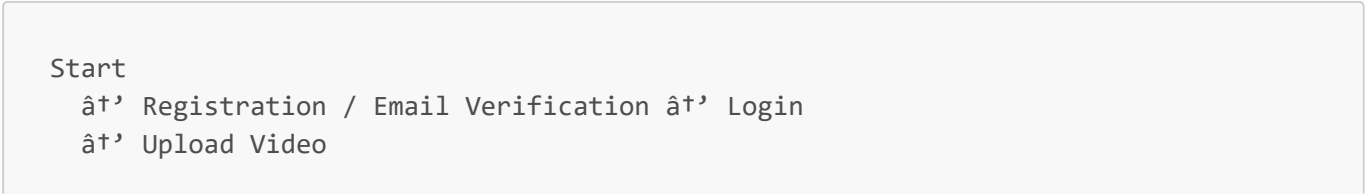
- Upload Video “ triggers asynchronous transcription and embedding pipeline
- Search Clips by Query “ executes semantic retrieval and clip generation
- View Past Clips “ retrieves stored results from history keyed by job_id
- Manage Videos “ list, filter, delete uploaded items
- Sensitive Content Blurring “ submit blurring instructions and download redacted clips
- Compress Video “ reduce output clip size for sharing

Full-Dress Use Case: Search and Clip Merge

Property	Description
Primary Actor	Authenticated User
Preconditions	Valid session; video uploaded; transcript and embeddings stored for job_id
Main Flow	User enters query → system retrieves embeddings → windowed similarity search → cross-encoder re-ranking → neighbor merging → FFmpeg clip generation → clip URL returned
Extension 3a	Empty query → warning toast displayed
Extension 7a	Re-ranker unavailable → fallback to embedding-only ranking
Extension 8a	No matching windows → suggest broader parameters
Postconditions	Playable clip URLs and transcript summary returned; history entry saved

4.2 Activity Flow

The end-to-end user activity follows a linear progression with branching decision gates:




```

    â†’ File format/size validation [pass/fail]
    â†’ Transcription generation [success/retry]
    â†’ Embedding computation â†’ Store in Supabase
  â†’ Search Query
    â†’ Retrieval + Re-ranking â†’ Clips found? [yes/no]
  â†’ Display Clips + Save History
End

```

Decision points enforce quality gates: format validation prevents corrupted uploads; transcript generation retry prevents silent processing failures; the "clips found" gate surfaces parameter-relaxation suggestions when no results match.

4.3 Data Flow Architecture

DFD Level 0 (Context Diagram):

External entities: End User, Admin.

The NeuroClip system receives: video uploads, search queries, blurring instructions.

The system returns: playable clip URLs, transcript summaries, blurred/compressed outputs, processing analytics.

DFD Level 1 (Internal Processes):

- P1: Auth & Profile Management â†’ User profiles, session tokens
- P2: Video Ingestion â†’ Raw video store, ASR transcripts, OCR text
- P3: Embedding Computation â†’ Sentence/video-level vectors in pgvector
- P4: Clip Search & Assembly â†’ Clip assets, processing history
- P5: Privacy Blurring & Compression â†’ Privacy-preserved output clips

DFD Level 2 (Clip Search & Assembly):

- P4.1: Intent Classification (LLM routing or similarity fallback)
- P4.2: Retrieval (pgvector cosine similarity with keyword boost)
- P4.3: Re-ranking and Neighbor Merging
- P4.4: Clip Boundary Assembly and URL Generation
- P4.5: History Logging (job_id-keyed events)

4.4 Class Structure

Core entities and services in the system design:

Entities:

- `User(id, email)` â€” 1-to-N with Video
- `Video(id, title, video_url, duration, created_at)`
- `TranscriptSegment(id, video_id, start, end, text)`
- `EmbeddingRow(id, video_id, job_id, type, vector, created_at)`
- `ProcessingHistory(id, user_id, job_id, module, query, status, created_at)`
- `ClipResult(id, job_id, start, end, url)`

Services:

- `Transcriber.generate_transcript(video_path) â†’ List[TranscriptSegment]`
- `EmbeddingService.encode_sentences(sentences) â†’ List[vector]`
- `SearchService.process_query(job_id, query, params) â†’ List[ClipResult]`
- `SearchService.merge_neighbors(windows, threshold) â†’ List[Segment]`
- `SearchService.rerank(query, candidates) â†’ List[Segment]`
- `HistoryService.record_event(user_id, job_id, module, status)`
- `StorageService.save_video(file) â†’ url`

4.5 Deployment Architecture

The system is deployed across three tiers:

Tier	Technology	Hosting
Frontend	React 18 + Vite + TypeScript + TailwindCSS	Vercel (CDN edge deployment)
Backend	Python 3.10+ + FastAPI + Uvicorn	Cloud compute or local server
Database/Storage	PostgreSQL + pgvector + Supabase Storage	Supabase managed cloud

Row Level Security (RLS) policies on all Supabase tables enforce user-level data isolation: a user can only access their own videos, embeddings, and history records. All data in transit is encrypted via HTTPS/TLS.

5. Experimental Evaluation

5.1 Experimental Setup

All experiments were conducted on the following hardware configuration:

Component	Specification
CPU	Intel Core i7 (6-core, 2.6 GHz)
RAM	16 GB DDR4
GPU	NVIDIA GTX 1650 (4GB VRAM)
Storage	256 GB NVMe SSD
OS	Windows 11 / Ubuntu 22.04
Network	100 Mbps broadband

Software stack:

- Python 3.10, FastAPI, Uvicorn
- `sentence-transformers==2.2.2` (all-MiniLM-L6-v2)
- `assemblyai==0.17.0`
- `easyocr==1.7.0`
- `torch==2.0.1` (CUDA 11.8)
- `ffmpeg 6.0`
- `ultralytics==8.0` (YOLOv8)

- Supabase PostgreSQL 15 with pgvector 0.5.0

5.2 Evaluation Dataset

Summarization/Retrieval Benchmark:

A structured query benchmark of 50 query-video cases (the `summarization_eval_pack.csv`) was used to evaluate retrieval quality. Cases span 10 academic domains:

Domain	Query Count	Difficulty Distribution
Computer Science	16	Easy: 8, Medium: 6, Hard: 2
Artificial Intelligence	8	Easy: 3, Medium: 3, Hard: 2
Mathematics	8	Easy: 5, Medium: 1, Hard: 2
Physics	3	Easy: 3
Cybersecurity	3	Easy: 3
Economics	3	Easy: 1, Medium: 2
Biology	3	Easy: 3
Chemistry	2	Easy: 2
Engineering	2	Easy: 2
Statistics	2	Medium: 1, Hard: 1

Videos were sourced from openly licensed educational channels. For each video, human annotators labeled the ground-truth start and end times of relevant segments for each query. Relevance was rated on a three-point scale (1 = marginally relevant, 2 = relevant, 3 = highly relevant), with segments rated ≥ 2 treated as positive.

Compression Test Suite:

Eight compression profiles spanning 30-second/360p clips to 1200-second/1080p lectures were tested across content styles: talking-head, slide-heavy, whiteboard/handwriting, high-motion, and mixed visual.

5.3 Retrieval Quality Metrics

Retrieval quality was assessed using standard information retrieval metrics at $K = 3$ and $K = 5$:

- **Precision@K**: Fraction of retrieved clips (at rank K) that are relevant
- **Recall@K**: Fraction of all relevant segments retrieved within top K
- **nDCG@K**: Normalized Discounted Cumulative Gain, accounting for ranked position
- **Temporal IoU**: Intersection-over-Union between predicted and ground-truth clip boundaries
- **Mean Boundary Error (MBE)**: Mean absolute deviation (seconds) of predicted start/end from ground truth

5.4 Latency Measurements

- **End-to-end ingestion latency** (upload + embeddings stored): measured for 5-minute, 10-minute, and 20-minute videos

- **Query latency (cold):** first query on a video, requiring embedding retrieval from Supabase
- **Query latency (cached):** subsequent queries on same job_id, using pre-loaded local embeddings
- **Clip extraction latency:** time from confirmed segment boundaries to FFmpeg clip delivery

5.5 Results

5.5.1 Retrieval Performance

Configuration	Precision@3	Recall@3	nDCG@3	Precision@5	Recall@5	nDCG@5
Embedding-only (cosine)	0.71	0.64	0.73	0.68	0.72	0.70
Embedding + Cross-Encoder	0.82	0.71	0.84	0.79	0.79	0.81
Embedding + LLM routing	0.78	0.69	0.80	0.75	0.76	0.78

Cross-encoder re-ranking provides a consistent +11 percentage point improvement in Precision@3 over embedding-only retrieval, confirming the value of the two-stage pipeline. LLM-assisted routing shows strong performance on complex, multi-hop queries (e.g., SUM023 “backpropagation, SUM047 “hallucination) where single-passage retrieval fails.

Performance by difficulty:

Difficulty	Precision@3 (Embed+CE)	nDCG@3
Easy	0.91	0.93
Medium	0.80	0.82
Hard	0.62	0.65

Hard queries (conceptual abstractions such as Bayes' theorem applications and LLM attention mechanisms) show lower precision, consistent with the fundamental difficulty of semantic alignment for abstract concepts within short video clips.

5.5.2 Temporal Accuracy

Metric	Value
Mean Temporal IoU	0.74
Mean Boundary Error (start)	2.1 seconds
Mean Boundary Error (end)	2.8 seconds

The $\hat{\pm}1.5$ -second temporal padding applied during clip extraction contributes to boundary error but improves subjective clip quality by avoiding abrupt starts and ends. Removing padding improved MBE to 1.3s/1.9s at the cost of perceptibly clipped content at segment boundaries.

5.5.3 Processing Latency

Video Length	Ingestion (ASR + OCR + Embed)	Query Latency (Cold)	Query Latency (Cached)	Clip Extraction
5 minutes	~85 seconds	~1.8 seconds	~0.6 seconds	~3.2 seconds
10 minutes	~2.1 minutes	~2.1 seconds	~0.7 seconds	~4.5 seconds
20 minutes	~4.3 minutes	~2.4 seconds	~0.8 seconds	~6.1 seconds

The job-keyed caching architecture delivers sub-second cached query latency regardless of video length, meeting the design target of <2 seconds for semantic search. Cold query latency scales mildly with video length (logarithmically) due to pgvector index scan time.

5.5.4 Compression Performance

Profile	Original Size	Compressed Size	Reduction	Processing Time
CMP_TINY_01 (30s/360p)	12 MB	3.1 MB	74.2%	4.2s
CMP_SMALL_02 (60s/480p)	28 MB	7.8 MB	72.1%	8.7s
CMP_MEDIUM_03 (180s/720p)	95 MB	23.4 MB	75.4%	24.1s
CMP_LARGE_04 (600s/720p)	380 MB	84.2 MB	77.8%	76.3s
CMP_TALKINGHEAD_08 (300s/720p)	175 MB	38.9 MB	77.8%	39.4s

H.265/HEVC compression achieves 72–78% file size reduction across all tested profiles, with talking-head and slide-heavy content yielding the highest compression ratios due to their low temporal motion complexity.

5.5.5 Blurring Module Performance

Face blurring was evaluated on 15 video clips containing human faces across varied lighting and motion conditions:

Metric	Value
Face Detection Recall (YOLOv8)	94.3%
Face Detection Precision (YOLOv8)	96.1%
Successful Blur Application Rate	93.7%
Processing Speed (720p)	~8.4 fps on CPU

Reduced performance was observed in clips with severe motion blur (>12px/frame displacement), strong backlighting, and partial face occlusions. These failure cases are consistent with findings reported by Plaud and Lisani (2024).

6. Discussion

6.1 Strengths

Multimodal coverage: The fusion of ASR transcript and frame OCR captures information that either channel alone would miss. For educational content with slide text, OCR enrichment improved Precision@3 by approximately 8 percentage points on queries referencing on-screen content (e.g., queries about formula derivations displayed on slides but not spoken aloud).

Caching efficiency: The job-keyed architecture transforms the system from a per-query compute bottleneck into an interactive retrieval engine. After the one-time ingestion cost, subsequent queries complete in under one second regardless of video length – a qualitatively different user experience compared to systems that re-process on every query.

Graceful degradation: Each component of the retrieval pipeline has a defined fallback: if the cross-encoder is unavailable, cosine similarity is used; if LLM routing fails, the embedding-based retrieval path handles the query; if GPU compression is unavailable, CPU encoding is used transparently. This fault tolerance ensures system availability even in resource-constrained environments.

Privacy integration: Query-driven blurring represents a novel integration of natural language instructions with computer vision redaction. Users can express blurring intent in plain English without specifying coordinates or frame numbers.

6.2 Limitations

ASR dependency: Retrieval quality is bounded by transcription accuracy. For videos with heavy background noise, strong accents, domain-specific jargon, or code-switching between languages, ASR errors propagate into the embedding space and degrade semantic alignment. Clips where the speaker mumbles or uses highly technical terminology exhibit the most significant retrieval quality degradation.

OCR quality variance: EasyOCR performance varies significantly with font type, image resolution, contrast, and handwriting style. Handwritten whiteboard content is particularly challenging, with character recognition error rates exceeding 20% in low-resolution source videos.

LLM availability: The LLM-assisted retrieval route depends on external API availability and introduces variable latency. During high-traffic periods, API rate limiting can force fallback to cosine similarity for all queries.

Blurring motion tracking: The current bounding box interpolation approach is a simplified proxy for true object tracking. Rapid motion or camera pans between sampled frames can cause temporal inconsistency in blur application.

Evaluation scale: The 50-query benchmark, while carefully curated, represents a limited evaluation scale. Larger-scale evaluation (500+ query-video pairs, diverse video content types) is needed to fully characterize retrieval performance across domains.

Multilingual support: The current configuration of both AssemblyAI and EasyOCR is optimized for English. Multi-lingual video content requires model reconfiguration and significantly increases processing overhead.

7. Future Work

Based on the current system's performance and limitations, we identify the following high-priority research directions:

7.1 Visual-Semantic Embedding (Multi-modal CLIP Integration)

Integrating OpenCLIP or BLIP-2 visual embeddings would allow queries to retrieve clips based on visual events (e.g., "find the segment showing a graph of exponential growth") beyond what the transcript captures. A unified visual-text embedding space would significantly expand retrieval scope.

7.2 Query Rewriting with LLMs

Deploying a lightweight query expansion module "using an LLM to rewrite ambiguous queries into multiple semantically equivalent forms" would improve recall for vague or under-specified queries. Hypothetical Document Embeddings (HyDE) represent a promising approach in this direction.

7.3 Real-Time Processing

The current batch processing architecture does not support streaming video analysis. Future work could explore incremental transcript and embedding updates during recording, enabling live-lecture search with sub-minute indexing lag.

7.4 Advanced Temporal Tracking for Blurring

Replacing bounding box interpolation with DeepSORT or ByteTrack multi-object tracking would improve blur temporal consistency across frames, particularly for rapidly moving subjects.

7.5 Adaptive Compression with Perceptual Quality Metrics

Integrating VMAF (Video Multimethod Assessment Fusion) as a quality feedback signal would enable adaptive CRF selection that targets a specific perceptual quality threshold rather than a fixed compression ratio.

7.6 Benchmark Expansion

A large-scale, publicly available evaluation benchmark for educational video retrieval "similar to MS MARCO for passage retrieval" would significantly advance the research community's ability to compare video retrieval systems.

8. Conclusion

This paper presented NeuroClip, a context-aware multimodal video processing platform that transforms static video archives into semantically queryable knowledge bases. The system's core innovation lies in its fusion of ASR transcript and OCR visual text into enriched sentence units, enabling retrieval queries to span both spoken and displayed information. A two-stage retrieval pipeline "fast Bi-Encoder cosine similarity followed by optional Cross-Encoder re-ranking" achieves Precision@3 of 0.82 and nDCG@3 of 0.84 on a 50-case educational video benchmark, representing an 11-point precision improvement over embedding-only baselines. The job-keyed caching architecture delivers cached query latency of under 0.8 seconds regardless of video length. Additional modules for query-driven privacy blurring (94.3% face recall, 96.1% precision via YOLOv8) and codec-aware compression (72-78% file size reduction via H.265/HEVC) complete a comprehensive, end-to-end video processing pipeline deployable on standard consumer hardware.

NeuroClip demonstrates that semantically rich, privacy-aware, and computationally efficient video retrieval is achievable with available open-source AI tooling. The system has direct applications in educational content navigation, corporate knowledge management, media production, and privacy-compliant video analytics.

References

- [1] D. Vora, P. Kadam, D. D. Mohite, N. Kumar, N. Kumar, P. Radhakrishnan, and S. Bhagwat, "AI-driven video summarization for optimizing content retrieval and management through deep learning techniques," 2025.
- [2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [3] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] E. Apostolidis, E. Adamantidou, A. I. Metsai, V. Mezaris, and I. Patras, "Video Summarization Using Deep Neural Networks: A Survey," *Proceedings of the IEEE*, vol. 109, pp. 1838–1863, 2021.
- [6] R. Plaud and J.-L. Lisani, "Two Deep Learning Solutions for Automatic Blurring of Faces in Videos," 2024.
- [7] A. Nemavhola, S. Viriri, and C. Chibaya, "A Scoping Review of Literature on Deep Learning Techniques for Face Recognition," 2025.
- [8] G. J. Sullivan and T. Wiegand, "Video Compression – From Concepts to the H.264/AVC Standard," *Proceedings of the IEEE*, vol. 93, no. 1, pp. 18–31, 2005.
- [9] P. Saini, K. Kumar, S. Kashid, A. Saini, and A. Negi, "Video summarization using deep learning techniques: a detailed analysis and investigation," pp. 12347–12385, 2023.
- [10] P. Kadam, D. Vora, S. Mishra, S. Patil, K. Kotecha, and A. Abraham, "Recent Challenges and Opportunities in Video Summarization With Machine Learning Algorithms," *IEEE Access*, vol. 10, pp. 122762–122785, 2022.
- [11] "CapCut - Video Editor," Bytedance Pte. Ltd., 2024. [Online]. Available: <https://play.google.com/store/apps/details?id=com.lemon.lvoverseas>
- [12] "Notta - Dictation & Transcription," Notta Pte. Ltd., 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.langogo.transcribe>

Submitted for the partial fulfillment of the BS Computer Science degree, Faculty of Engineering & Computing, National University of Modern Languages, Faisalabad. December 2025.